

Overview

The learning outcomes for Assignment 2 are:

- Define and discuss quantitative metrics to critically assess common quality attributes of an existing software system.
- Critically evaluate a software architecture based on its documentation to assess how well the desired qualities can be achieved.

Read the whole assignment text. Markers  indicate the section of your report in which to put your work results. The required overall report structure is also summarized at the end of this document.

1 Preparation

1.1 Task: Select and Explore a Software System

Select a software system for your analysis that satisfies the following criteria:



- Source code publicly available (e.g., on GitHub or another hosting service).
- Has commit history of 2+ years (first commit to last commit).
- Has 1,000+ commits.
- Has 4+ contributors with 50+ commits. (Otherwise: Discuss the implications!)
- Is not [lxml](#) (because we covered it in Workshop 2 already)

Include the following aspects in  **1.1 System Description & History**:

- Describe the system: What is it for (think: primary functionality)? Who are its users (groups)?
- Provide a link to the system's source code.
- Provide the following key characteristics (e.g., numbers or simple graphs): How old is the project? How big is the software (e.g., Lines of Code)? How many people contributed to its source? How did the project activity change over time?



You may want to choose a software system you have used yourself, or maybe even contributed to in the past. These are some of the software systems that students analyzed in previous years: [Pandas](#), [NumPy](#), [GIMP](#), [OWASP ZAP](#), [Wordpress](#), [Bitwarden](#), [Homebrew](#), [Mastodon](#)

1.2 Task: Decide on Relevant Quality Attribute

You will assess three quality attributes of the selected system: *Modifiability* and *Testability* (because they are almost universally relevant) and then a third quality attribute of your choice. Identify which other quality attribute is relevant for your particular system and provide the following in  **1.2 Quality Attributes**:

- Give a brief definition for each of the three quality attributes.
- Discuss the importance of *Modifiability* and *Testability* for your system (or parts of it).
- Provide a rationale for selecting the third quality attribute for your system (or parts of it).

1.3 Task: Identify Architectural Decisions

For an in-depth analysis of the software system, you first need to understand some of its architecture decisions. Identify and document architecturally relevant decisions for all three quality attributes.

To be able to assess *Modifiability* (and to some degree *Testability*, too), get an overview of the system's module structure [SAP, Sec. 1.2]. Some systems adopt a well-known architecture style (such as *Layers*, *Pipeline*, *Microkernel*, etc., [FSA, Pt. II]), others are split in more idiosyncratic modules. Some projects explicitly document such decisions outside of their source code (Jenkins: [Layer Architecture](#) is a good example), in many other cases you need to infer them from the source code: Look at the top-level packages or naming conventions of files, for example.

For the third quality attribute, also try to identify a pattern or tactic in the system that is meant to support that attribute (e.g., maybe there is a *Cache* somewhere for *performance*, or *Encryption* for *security*, etc.).

In **§ 1.3 System Structure & Architecture Decision**, document relevant architecture decision by providing at least the following:

- A *decomposition view* [DoSA, Sec. 2.1] that focuses on the productive code (i.e., no test code, no documentation, or other non-code) with an appropriate granularity such that you get 3 to 9 modules. Add relationships between the modules (e.g., <<allowed to use>>) if they are part of the design decision.
- Another *view* see [DoSA, Ch. 2, 4], or 5 to document the pattern or tactic to support the third quality attribute.
 - If you cannot identify such a pattern or tactic, instead document this absence and reflect on the consequences.
- Briefly explain how you identified the architecture decisions (e.g., ‘from documentation’ or ‘inferred from source code’).

2 Analysis

You may use the tools showcased in Workshops 2 and 3 (i.e., [GitTruck](#) and [CodeScene](#)) and you are free to use additional tools and additional data sources – be creative! It's also okay to use GenAI to code up some scripts for data collection and analysis (refer to [GenAI Policy](#), Sec. 2.4). Keep in mind that your ultimate goal is to evaluate the software system regarding the three selected quality attributes. Any metric you select from the tools must be in service of achieving that goal.

2.1 Task: Select Appropriate Metrics

No single metric can fully represent a quality attribute. For each of your three attributes, select **three metrics** that can provide useful insights.

In the “Metrics” sections of the report **§ 2.1, 3.1, 4.1** (i.e., for each quality attribute), include the following for each of the three respective metrics:

- A short definition: What does the metric measure, and how is it obtained?
- A justification: Why does a higher (or lower) value indicate higher quality?
- A limitation: In what situation could the metric be misleading?

💡 Be creative in your choice of metrics. Remember that there are different type (structural, process, or operational [FSA, Ch. 6]), as there are types of analyses (e.g., explorative analysis to identify hotspots, or comparative analyses against the project's own past to identify trends, against a specific reference value, e.g., from a *quality attribute scenario*, or against any other “fitness function”; see [Lecture 6](#) and [FSA, Ch. 6]).

2.2 Task: Do the Measurements

Include the following in the respective “Measurements” sections of the report **§ 2.2, 3.2, 4.2**:

- For each metric, a description of how you collected the data: Which tool did you use? On which version of the source code (or time span, for process metrics)?
- Present your measurement, e.g., numbers, tables, or graphs.

2.3 Task: Discuss Results

Include the following in the respective “Discussion” sections of the report **2.3, 3.3, 4.3**:

- What do the metrics say about the quality attribute overall?
- Do the different metrics point in the same direction, or are there conflicts? If they conflict: How can this be explained?
- How do the findings relate to the identified architecture decisions? Is the decision effective? Is it followed by the developers?
- If problems exist, what would improve the situation: A different architecture decision, or better enforcement of the existing one?

2.4 Task: Ensure Analysis Reproducibility

All data and analysis artifacts produced or collected for this assignment must be made available in a publicly accessible Git repository. This includes, as applicable:

- downloaded or extracted raw data, such as CSV or JSON files;
- scripts used to collect, process, or analyze the data;
- configuration files and relevant tool settings; and
- intermediate data necessary to understand or reproduce the reported measurements.

The repository may be hosted on any publicly accessible Git hosting service, such as GitHub, GitLab, or Codeberg.

In the **Introduction** of your report, provide:

- the URL of the repository; and
- the full Git commit ID that identifies the exact version of the repository corresponding to the submitted report.

The specified commit must remain publicly accessible at least until the assignment has been graded. Organize and document the repository sufficiently for a reviewer to identify which artifacts correspond to the measurements reported in Sections **2, 3, 4**.

References

- DoSA** P. Clements *et al.*, *Documenting Software Architectures: Views and Beyond*, 2nd ed. Addison-Wesley Professional, 2010. Available: <https://learning.oreilly.com/library/view/documenting-software-architectures/9780132488617/>
- FSA** M. Richards and N. Ford, *Fundamentals of Software Architecture: A Modern Engineering Approach*, 2nd ed. O'Reilly Media, Inc., 2025. Available: <https://learning.oreilly.com/library/view/fundamentals-of-software/9781098175504/>
- SAP** L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley Professional, 2021. Available: <https://learning.oreilly.com/library/view/software-architecture-in/9780136885979/>
-

Submission

To complete this assignment, you need to submit a report. You are expected to work on this as a group, i.e., you should discuss your metrics choice and the implications of your measurements. You can, of course, split up the work of finalizing individual sections, but ensure that the final report is a *coherent* whole.

Please structure your report as follows:

- **Introduction**
 - List of group members who contributed to this report.
 - Statement of AI usage according to course's [GenAI Policy](#), Section 3.1.
 - Data availability statement containing the public Git repository URL and the full commit ID corresponding to the submitted report.
- **Section 1: Overview**
 - **Section 1.1: System Description & History**
 - **Section 1.2: Quality Attributes**
 - **Section 1.3. System Structure & Architecture Decisions**
- **Sections 2, 3, 4: Modifiability, Testability, <Your Quality Attribute>**
 - **Section x.1: Metrics**
 - **Section x.2: Measurements**
 - **Section x.3: Discussion**

There is no formal page limit, but try to keep the report as short as possible, e.g., between 8 and 15 pages.

i **Tip:** You can find a [LaTeX template here](#), and we recommend using it.

Grading

Your report has to be submitted to Canvas as one PDF file before the deadline. It will then be graded according to the criteria and procedures defined in the [Rubric](#) (published alongside this assignment in Canvas).